

Лабораторна робота № 3

Node.js, Promise, MySQL

Завдання 1.

Студентам надається готовий програмний проєкт Lab03 (далі — вихідний проєкт).

- Ваше завдання полягає у проведенні його аналізу, дослідженні логіки роботи та подальшій реалізації власної версії цього проєкту з використанням Node.js.
- Нова реалізація повинна зберігати основний функціонал вихідного проєкту, але може відрізнятися структурою коду, підходами до реалізації та оптимізацією окремих частин.

У результаті виконання завдання студент повинен надати:

- Репозиторій або архів з реалізованим проєктом.
- Короткий технічний опис (звіт), який містить:
 - опис структури проєкту;
 - пояснення використаних технологій;
 - аналіз змін порівняно з початковим проєктом;
 - обґрунтування рішень щодо оптимізації або спрощення.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.122.23.000 – Лр.3		
Змн.	Арк.	№ докум.	Підпис	Дат			
Розроб.		Шевчук Е. Л.					
Перевір.		Ковтун В. В.					
Реценз.							
Н. Контр.							
Зав.каф.							
					Літ.		
					Арк.		
					Аркушів		
					1		
					ФІКТ, гр. КН23-3		

1. Структура проекту

Проект організовано за принципом MVC (Model–View–Controller) з додатковими шарами сервісів, middleware та маршрутизації. Нижче наведено повну файлову структуру:

```
lab3/
├── app.js                # Точка входу: Express, middleware, маршрути, Socket.io
├── package.json          # Залежності та скрипти запуску
├── config/
│   ├── auth.js          # Конфігурація express-session
│   └── db.js             # Пул з'єднань MySQL через mysql2/promise
├── controllers/
│   ├── authController.js # Логін, логіт (робота з сесією)
│   ├── userController.js # CRUD-операції над користувачами, dashboard
│   └── contactController.js # CRUD-операції над контактами
├── middleware/
│   ├── authMiddleware.js # authOnly, adminOnly, guestOnly — захист маршрутів
│   └── validate.js       # Joi-валідація тіла запиту
├── models/
│   ├── User.js          # SQL-запити до таблиці users
│   └── Contact.js       # SQL-запити до таблиці contacts
├── routes/
│   ├── web/
│   │   ├── auth.js      # GET/POST /login, /logout, /dashboard
│   │   ├── users.js     # CRUD-маршрути /users/*
│   │   └── contacts.js  # CRUD-маршрути /contacts/*
│   └── api/
│       ├── apiUsers.js  # REST API /api/users
│       └── apiContacts.js # REST API /api/contacts
├── schemas/
│   └── userSchema.js     # Joi-схема валідації даних користувача
├── services/
│   ├── userService.js   # Бізнес-логіка: хешування, перевірка email, логи
│   ├── contactService.js # Бізнес-логіка контактів
│   └── logService.js     # Запис подій до таблиці logs
├── sockets/
│   └── userSocket.js     # Socket.io: real-time сповіщення про дії з юзерами
├── views/
│   ├── layout.ejs       # Головний layout (header, nav, <%- body %>)
│   ├── dashboard.ejs
│   ├── auth/login.ejs
│   ├── users/           # list, details, create, edit
│   └── contacts/        # list, form
├── public/
│   ├── css/style.css    # Toast-сповіщення та підсвітка рядків
│   └── js/app.js        # Клієнтський JS: Socket.io, toast, підтвердження
└── user_management_pro.sql # Дамп MySQL: таблиці users, contacts, logs
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.122.23.000 – Лр.2	Арк.
Змн.	Арк.	№ докум.	Підпис	Дат		2

2. Використані технології

Node.js

Однопотокowe середовище виконання JS на сервері. Завдяки Event Loop Node.js обробляє I/O-операції (запити до БД, мережа) неблокуючим способом: поки очікує відповідь від MySQL, він обробляє інші запити. Це робить сервер ефективним навіть при великій кількості одночасних з'єднань без потреби у потоках (threads).

Promises та async/await

Promises — об'єкти, що представляють результат асинхронної операції (pending → fulfilled/rejected).
async/await — синтаксичний цукор над Promises, що дозволяє писати асинхронний код у лінійному стилі без "callback hell".

Ключова оптимізація — Promise.all(): запускає кілька незалежних промісів паралельно, замість послідовного виконання. Це безпосередньо пов'язано з Event Loop — обидва запити до БД потрапляють у чергу подій одночасно, не блокуючи один одного.

```
// Послідовно (оригінал) — сумарний час: ~t1 + t2
const user = await userService.getUserById(id); // чекаємо...
const contacts = await contactService.getContacts(id); // тільки потім
```

```
// Паралельно (нова версія) — сумарний час: ~max(t1, t2)
const [user, contacts] = await Promise.all([
  userService.getUserById(id),
  contactService.getContacts(id)
]);
```

MySQL + mysql2/promise

mysql2/promise надає нативну підтримку Promises для пулу з'єднань MySQL. Пул (createPool) автоматично управляє кількома з'єднаннями, що дозволяє обслуговувати паралельні запити без очікування звільнення одного з'єднання. Запити виконуються через параметризовані placeholders (?), що захищає від SQL-ін'єкцій.

3. Аналіз змін: нова версія vs оригінал

У таблиці нижче описано виявлені проблеми вихідного проєкту та відповідні виправлення:

#	Файл	Проблема в оригіналі	Виправлення
1	authController.js	Критичний баг: пароль порівнювався як рядок (password !== user.password). Після bcrypt.hash() при реєстрації вхід був неможливий	Замінено на bcrypt.compare(password, user.password)
2	routes/web/users.js	Баг маршрутизації: GET /:id реєструвався до /edit/:id і /delete/:id.	Специфічні маршрути розміщені до параметричного /:id
Змн.	Арк.	№ докум.	Підпис
Житомирська політехніка. 26.12.23.000 – лр.2			
Експерт перекоплював рядки "edit" і			
Арк. 3			

#	Файл	Проблема в оригіналі	Виправлення
		"delete" як ID	
3	routes/web/contacts.js	Та сама проблема: GET /:userId поглинав /edit/:id	Виправлено аналогічно — специфічні маршрути вище
4	routes/web/auth.js	Dashboard-логіка дублювалась прямо в маршруті, ігноруючи сервісний шар	Перенесено до userController.dashboard
5	db/database.js	Мертвий код: файл ніде не імпортувався, вказував на іншу БД (user_demo)	Файл видалено
6	userService.js	User.findAll() і User.count() виконувались послідовно, хоча не залежать одне від одного	Замінено на Promise.all([...])
7	userController.details	getUserById і getContacts — послідовні незалежні запити	Замінено на Promise.all([...])
8	contactController.list	getUserById і getContacts — послідовні незалежні запити	Замінено на Promise.all([...])
9	app.js	Відсутній глобальний обробник помилок — необроблені помилки в асупс-хендлерах зависали або крешили процес	Додано app.use((err, req, res, next) => {...})
10	routes/api/*.js	Відсутній try/catch у всіх async API-маршрутах — помилки БД не оброблялись	Додано try/catch + next(err) у кожен хендлер

4. Обґрунтування рішень

Чому bcrypt.compare() є обов'язковим

bcrypt.hash() — однобічна функція: відновити оригінальний пароль з хешу неможливо. Тому єдиний спосіб перевірки — bcrypt.compare(plaintext, hash), яка внутрішньо хешує введений пароль і порівнює результати. Пряме порівняння рядків (===) — це груба помилка безпеки.

Чому важливий порядок маршрутів в Express

Express перевіряє маршрути зверху вниз у порядку реєстрації. Параметричний маршрут /:id — жадібний: він збігається з будь-яким рядком, включно з "edit" і "delete". Якщо /:id зареєстровано першим, запит GET /users/edit/5 обробляється як id = "edit", а не як маршрут редагування. Специфічні маршрути завжди мають бути вище параметричних.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.122.23.000 – Лр.З	Арк.
Змн.	Арк.	№ докум.	Підпис	Дат		4

Чому Promise.all() пришвидшує роботу

Event Loop Node.js — однопоточковий, але I/O (запити до БД) виконується асинхронно. При послідовному await другий запит починається тільки після завершення першого. Promise.all() передає обидва запити до черги подій одночасно: MySQL обробляє їх паралельно, і загальний час очікування дорівнює часу найповільнішого запиту, а не їх сумі.

Чому глобальний error handler необхідний

У Express 4+ помилки, кинуті в асинх-функціях (без явного next(err)), не передаються до глобального обробника автоматично — сервер зависає без відповіді клієнту. Глобальний (err, req, res, next) обробник забезпечує єдину точку обробки всіх непередбачених помилок і коректну відповідь 500.

Чому видалено db/database.js

Файл підключав іншу базу даних (user_demo) і ніде не імпортувався. Мертвий код збільшує когнітивне навантаження при читанні проєкту та може ввести в оману — наприклад, призвести до використання неправильного підключення при рефакторингу.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.122.23.000 – Лр.З	Арк.
Змн.	Арк.	№ докум.	Підпис	Дат		5